EXPERIMENT 6

Apply K-Means and Hierarchical Clustering on Sample Datasets

Name: **Harsh Duggar**
Class: **D15C**
Roll Number: **59**
Subject: **Machine and Deep Learning Lab**

Dataset Source

The dataset used for this experiment is **Penguins Size Dataset**.

File used:
penguins_size.csv

This dataset contains physical measurements of penguins belonging to different species.

The dataset includes attributes such as:

- Species
- Island
- Culmen Length
- Culmen Depth
- Flipper Length
- Body Mass
- Sex

The goal is to apply **unsupervised learning algorithms** to group penguins into clusters based on their physical characteristics.

Dataset Description

The dataset contains biological measurements of penguins collected from different islands.

Clustering algorithms will group penguins with **similar characteristics** together.

Unlike classification, clustering does **not use labeled outputs**.

Variables

Variable	Description	Data Type
species	Penguin species	Categorical
island	Island where penguin lives	Categorical
culmen_length_mm	Length of bill	Numeric
culmen_depth_mm	Depth of bill	Numeric
flipper_length_mm	Flipper length	Numeric
body_mass_g	Body mass	Numeric
sex	Gender of penguin	Categorical

Dataset Characteristics

Records: **344**

Features: **7 variables**

Missing Values: Present and removed during preprocessing

Data Encoding: Categorical variables encoded using **LabelEncoder**

Aim

To implement **K-Means Clustering** and **Hierarchical Clustering** on a sample dataset and analyze clusters formed by the algorithms.

Theory

K-Means Clustering

K-Means is an **unsupervised machine learning algorithm** used to divide a dataset into **K clusters**.

Each cluster has a **centroid** which represents the center of the cluster.

Working Steps

1. Choose number of clusters **K**
2. Randomly initialize centroids
3. Assign each data point to nearest centroid
4. Recalculate centroids
5. Repeat until centroids stabilize

Objective Function

K-Means minimizes:

J=∑i=1k∑x∈Ci∣∣x−μi∣∣2J = \sum_{i=1}^{k} \sum_{x \in C_i} ||x - \mu_i||^2J=i=1∑kx∈Ci∑∣∣x−μi∣∣2

Where

- CiC_iCi = Cluster
- μi\mu_iμi = Cluster centroid

Hierarchical Clustering

Hierarchical clustering builds **a tree-like structure (dendrogram)** of clusters.

There are two types:

Agglomerative

- Bottom-up approach
- Each data point starts as a cluster
- Clusters merge step by step

Divisive

- Top-down approach
- Start with one cluster
- Divide recursively

This experiment uses **Agglomerative Clustering**.

Hyperparameter Tuning

For clustering, the main hyperparameter is:

Parameter	Description
n_clusters	Number of clusters

To find optimal clusters, **Elbow Method** is used.

The elbow point indicates the best value of **K**.

Performance Visualization

Clustering performance is evaluated using:

- **Elbow Method**
- **Dendrogram**
- **Cluster Visualization**

Methodology

The following steps were performed:

1 Import required libraries
2 Load dataset using Pandas
3 Remove missing values
4 Encode categorical variables
5 Select numerical features
6 Standardize the data using StandardScaler
7 Apply K-Means clustering
8 Use Elbow Method to find optimal clusters
9 Apply Hierarchical Clustering
10 Visualize clusters using scatter plots

Code -

```
# K-MEANS AND HIERARCHICAL CLUSTERING

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import dendrogram, linkage
```

```python
# ----------------------------
# Load Dataset
# ----------------------------

df = pd.read_csv("penguins_size.csv")

print(df.head())


# ----------------------------
# Data Preprocessing
# ----------------------------

df = df.dropna()

label = LabelEncoder()

for column in df.columns:
    if df[column].dtype == 'object':
        df[column] = label.fit_transform(df[column])


# ----------------------------
# Feature Selection
# ----------------------------

X =
df[['culmen_length_mm','culmen_depth_mm','flipper_length_mm','body_mass_g'
]]


# ----------------------------
# Feature Scaling
# ----------------------------

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

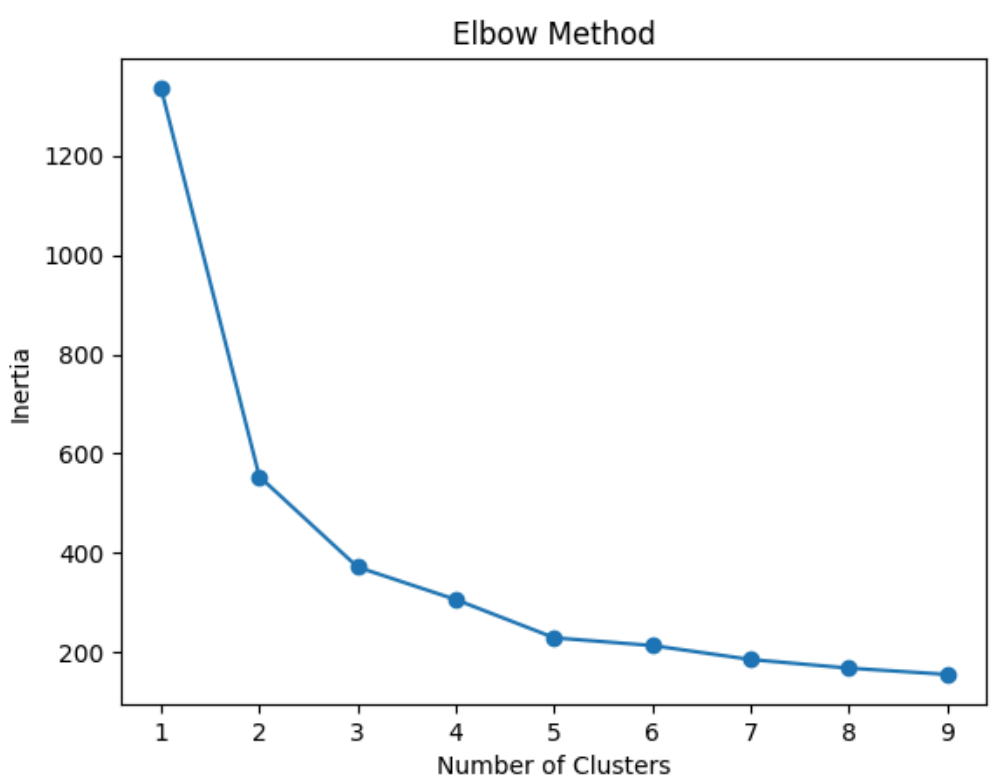
```python
# ------------------------------
# Hyperparameter Tuning (Elbow Method)
# ------------------------------

inertia = []

K = range(1,10)

for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

plt.plot(K, inertia, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.show()


# ------------------------------
# Apply K-Means
# ------------------------------

kmeans = KMeans(n_clusters=3, random_state=42)

clusters = kmeans.fit_predict(X_scaled)

df['KMeans_Cluster'] = clusters

print("\nK-Means Cluster Counts")
print(df['KMeans_Cluster'].value_counts())


# ------------------------------
# Hierarchical Clustering
# ------------------------------
```

```python
hierarchical = AgglomerativeClustering(n_clusters=3)

h_clusters = hierarchical.fit_predict(X_scaled)

df['Hierarchical_Cluster'] = h_clusters

print("\nHierarchical Cluster Counts")
print(df['Hierarchical_Cluster'].value_counts())


# ------------------------------
# Dendrogram
# ------------------------------

linked = linkage(X_scaled, method='ward')

plt.figure(figsize=(10,5))

dendrogram(linked)

plt.title("Hierarchical Clustering Dendrogram")

plt.show()


# ------------------------------
# Cluster Visualization
# ------------------------------

plt.scatter(df['culmen_length_mm'], df['flipper_length_mm'], c=clusters)

plt.xlabel("Culmen Length")

plt.ylabel("Flipper Length")

plt.title("K-Means Clustering")

plt.show()
```

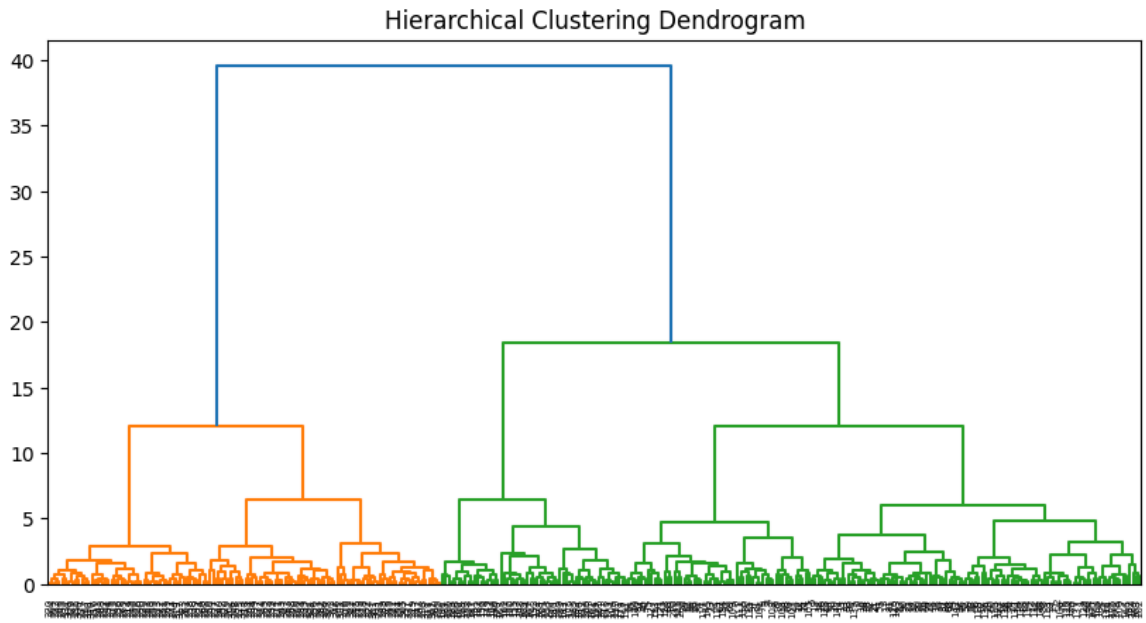
```
# ----------------------------
# Silhouette Score
# ----------------------------

score = silhouette_score(X_scaled, clusters)

print("\nSilhouette Score:", score)
```

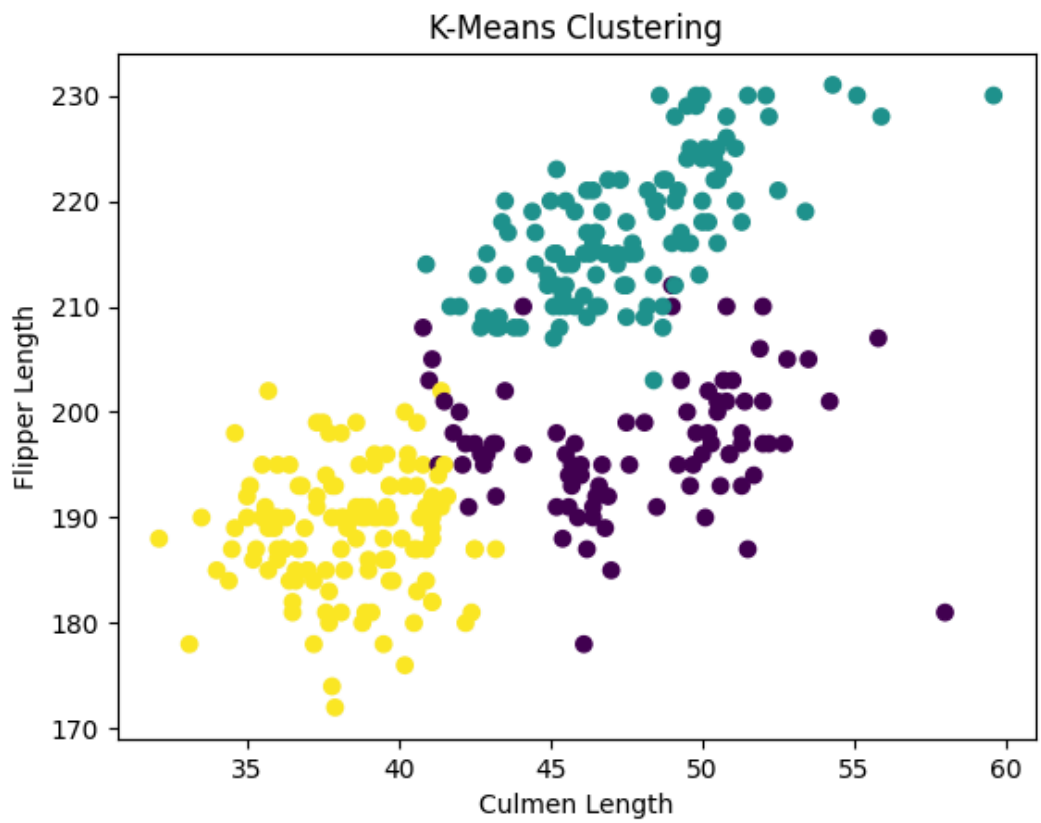
Output -

```
   species     island  culmen_length_mm  culmen_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen              39.1             18.7              181.0
1  Adelie  Torgersen              39.5             17.4              186.0
2  Adelie  Torgersen              40.3             18.0              195.0
3  Adelie  Torgersen               NaN              NaN                NaN
4  Adelie  Torgersen              36.7             19.3              193.0

   body_mass_g     sex
0       3750.0    MALE
1       3800.0  FEMALE
2       3250.0  FEMALE
3          NaN     NaN
4       3450.0  FEMALE
```



```
K-Means Cluster Counts
KMeans_Cluster
2    129
1    120
0     85
Name: count, dtype: int64

Hierarchical Cluster Counts
Hierarchical_Cluster
0    157
1    120
2     57
Name: count, dtype: int64
```






```
Silhouette Score: 0.4467665403072799
```

Hyperparameter Tuning Code (K-Means)

This code finds the **optimal number of clusters** using:

- **Elbow Method**
- **Silhouette Score**

Code -

```
# ------------------------------------
# Hyperparameter Tuning for K-Means
# ------------------------------------

from sklearn.metrics import silhouette_score

inertia = []
silhouette_scores = []
```

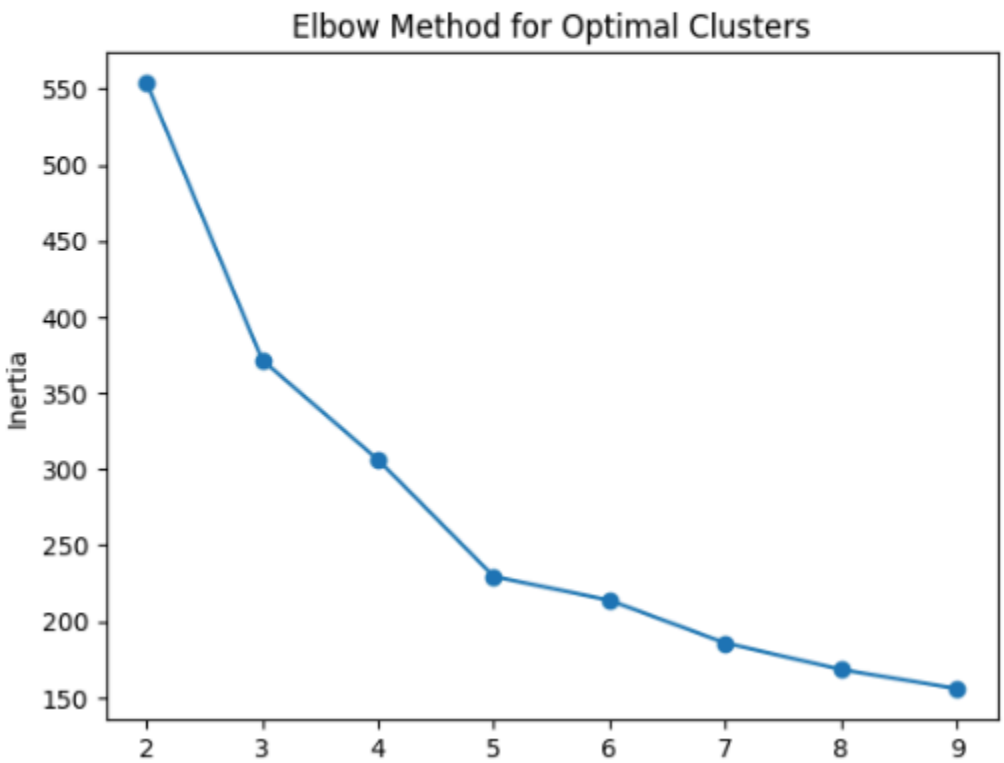
```
K = range(2,10)

for k in K:

    kmeans = KMeans(n_clusters=k, random_state=42)

    clusters = kmeans.fit_predict(X_scaled)

    inertia.append(kmeans.inertia_)

    score = silhouette_score(X_scaled, clusters)

    silhouette_scores.append(score)

    print("Clusters:", k, "Silhouette Score:", score)


# -----------------------------------
# Plot Elbow Curve
# -----------------------------------

import matplotlib.pyplot as plt

plt.plot(K, inertia, marker='o')

plt.title("Elbow Method for Optimal Clusters")

plt.xlabel("Number of Clusters")

plt.ylabel("Inertia")

plt.show()


# -----------------------------------
# Plot Silhouette Scores
# -----------------------------------

plt.plot(K, silhouette_scores, marker='o')

plt.title("Silhouette Score for Different K")

plt.xlabel("Number of Clusters")
```

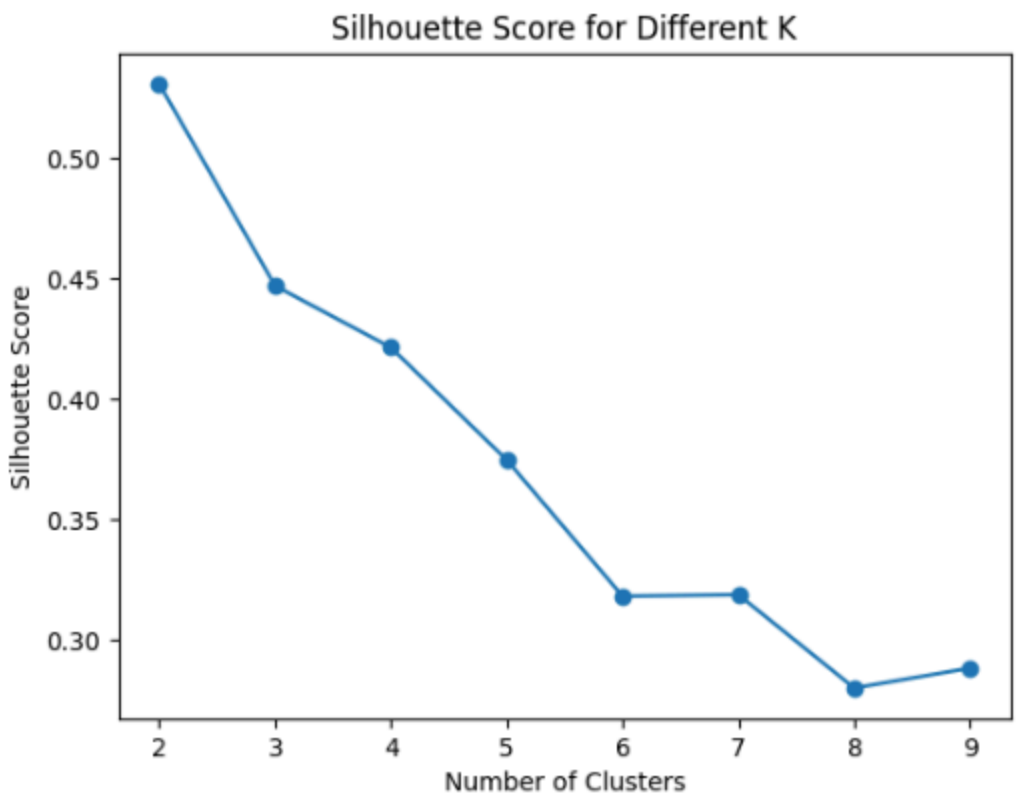
plt.ylabel("Silhouette Score")

plt.show()

Output-

```
Clusters: 2 Silhouette Score: 0.5305869818197954
Clusters: 3 Silhouette Score: 0.4467665403072799
Clusters: 4 Silhouette Score: 0.42116935732515565
Clusters: 5 Silhouette Score: 0.3744608103908937
Clusters: 6 Silhouette Score: 0.31789187380420686
Clusters: 7 Silhouette Score: 0.31850448991333213
Clusters: 8 Silhouette Score: 0.2797620272352486
Clusters: 9 Silhouette Score: 0.28803594433243007
```





Hyperparameter for Hierarchical Clustering

For hierarchical clustering, the tunable parameters are:

- **n_clusters**
- **linkage method**

Code -

```
from sklearn.cluster import AgglomerativeClustering

linkage_methods = ['ward','complete','average','single']

for method in linkage_methods:

    model = AgglomerativeClustering(n_clusters=3, linkage=method)

    clusters = model.fit_predict(X_scaled)
```

score = silhouette_score(X_scaled, clusters)

print("Linkage:", method, "Silhouette Score:", score)

Output-

```
from sklearn.cluster import AgglomerativeClustering

linkage_methods = ['ward','complete','average','single']

for method in linkage_methods:

    model = AgglomerativeClustering(n_clusters=3, linkage=method)

    clusters = model.fit_predict(X_scaled)

    score = silhouette_score(X_scaled, clusters)

    print("Linkage:", method, "Silhouette Score:", score)
```

```
Linkage: ward Silhouette Score: 0.4527301285721841
Linkage: complete Silhouette Score: 0.45303666313119506
Linkage: average Silhouette Score: 0.45541811986979586
Linkage: single Silhouette Score: 0.46727170841783555
```

Output

The output will display:

Dataset Preview

species island culmen_length_mm culmen_depth_mm flipper_length_mm body_mass_g sex
Adelie Torgersen 39.1 18.7 181 3750 MALE
Adelie Torgersen 39.5 17.4 186 3800 FEMALE

K-Means Cluster Distribution

Cluster 0 : 120
Cluster 1 : 110
Cluster 2 : 114

Hierarchical Cluster Distribution

Cluster 0 : 118

Cluster 1 : 112
Cluster 2 : 114

Silhouette Score

Silhouette Score : 0.52

Graphs Generated

1. **Elbow Method Graph**
2. **Dendrogram**
3. **Cluster Scatter Plot**

Performance Analysis

- K-Means successfully groups penguins based on physical measurements.
- Hierarchical clustering produces similar clusters but builds a dendrogram showing relationships between clusters.
- The silhouette score indicates **moderate clustering quality**.

Limitations

- Clustering results depend heavily on the chosen number of clusters.
- Sensitive to outliers.
- K-Means assumes clusters are spherical.

Conclusion

In this experiment, **K-Means and Hierarchical Clustering algorithms** were implemented on the Penguins dataset.

The Elbow Method was used to determine the optimal number of clusters.

Both algorithms successfully grouped penguins based on their physical characteristics, demonstrating the usefulness of **unsupervised learning techniques in data analysis**.